# cnn_mnist

June 16, 2025

[8]:
```
from tensorflow.keras.datasets import mnist
import warnings
import tensorflow as tf

warnings.filterwarnings('ignore')
tf.get_logger().setLevel('ERROR')

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

[9]:
```
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.layers import Dropout

# Common preprocessing for all CNNs

# Normalize pixel values
x_train_norm = x_train.astype('float32') / 255.0
x_test_norm = x_test.astype('float32') / 255.0

# Reshape for CNN input
x_train_cnn = x_train_norm.reshape(-1, 28, 28, 1)
x_test_cnn = x_test_norm.reshape(-1, 28, 28, 1)

# One-hot encode labels
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

[10]:
```
import time
import tracemalloc

# Helper function to measure time and memory
def measure_train_test(model, x_train, y_train, x_test, y_test, **fit_kwargs):
    tracemalloc.start()
    t0 = time.time()
    history = model.fit(x_train, y_train, **fit_kwargs)
    train_time = time.time() - t0
```

```
    train_mem, _ = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    tracemalloc.start()
    t0 = time.time()
    score = model.evaluate(x_test, y_test, verbose=0)
    test_time = time.time() - t0
    test_mem, _ = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    return score, train_time, train_mem/(1024*1024), test_time, test_mem/
 ↪(1024*1024)
```

[11]:
```
cnn1 = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

cnn1.compile(optimizer='adam', loss='categorical_crossentropy',␣
 ↪metrics=['accuracy'])
score1, train_time1, train_mem1, test_time1, test_mem1 = measure_train_test(
    cnn1, x_train_cnn, y_train_cat, x_test_cnn, y_test_cat, epochs=3,␣
 ↪batch_size=128, validation_split=0.1
)
print(f'CNN 1 Test accuracy: {score1[1]:.4f}')
print(f'CNN 1 Train time: {train_time1:.2f}s, Train peak mem: {train_mem1:.
 ↪2f}MB')
print(f'CNN 1 Test time: {test_time1:.2f}s, Test peak mem: {test_mem1:.2f}MB')
```

```
Epoch 1/3
422/422            7s 14ms/step -
accuracy: 0.8522 - loss: 0.5547 - val_accuracy: 0.9782 - val_loss: 0.0856
Epoch 2/3
422/422            6s 14ms/step -
accuracy: 0.9744 - loss: 0.0894 - val_accuracy: 0.9850 - val_loss: 0.0574
Epoch 3/3
422/422            6s 14ms/step -
accuracy: 0.9834 - loss: 0.0589 - val_accuracy: 0.9862 - val_loss: 0.0549
CNN 1 Test accuracy: 0.9828
CNN 1 Train time: 19.08s, Train peak mem: 2.45MB
CNN 1 Test time: 0.76s, Test peak mem: 0.09MB
```

[12]:
```
cnn2 = Sequential([
    Conv2D(64, (3,3), activation='relu', input_shape=(28,28,1)),
```

```
    MaxPooling2D((2,2)),
    Dropout(0.25),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])

cnn2.compile(optimizer='adam', loss='categorical_crossentropy',␣
 ↪metrics=['accuracy'])
score2, train_time2, train_mem2, test_time2, test_mem2 = measure_train_test(
    cnn2, x_train_cnn, y_train_cat, x_test_cnn, y_test_cat, epochs=3,␣
 ↪batch_size=128, validation_split=0.1
)
print(f'CNN 2 Test accuracy: {score2[1]:.4f}')
print(f'CNN 2 Train time: {train_time2:.2f}s, Train peak mem: {train_mem2:.
 ↪2f}MB')
print(f'CNN 2 Test time: {test_time2:.2f}s, Test peak mem: {test_mem2:.2f}MB')
```

```
Epoch 1/3
422/422             16s 35ms/step -
accuracy: 0.8261 - loss: 0.5624 - val_accuracy: 0.9783 - val_loss: 0.0772
Epoch 2/3
422/422             14s 34ms/step -
accuracy: 0.9620 - loss: 0.1309 - val_accuracy: 0.9837 - val_loss: 0.0588
Epoch 3/3
422/422             14s 34ms/step -
accuracy: 0.9726 - loss: 0.0894 - val_accuracy: 0.9867 - val_loss: 0.0479
CNN 2 Test accuracy: 0.9823
CNN 2 Train time: 44.85s, Train peak mem: 2.75MB
CNN 2 Test time: 1.13s, Test peak mem: 0.08MB
```

[13]:
```
cnn3 = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    Conv2D(32, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(256, activation='relu'),
    Dense(10, activation='softmax')
])

cnn3.compile(optimizer='adam', loss='categorical_crossentropy',␣
 ↪metrics=['accuracy'])
score3, train_time3, train_mem3, test_time3, test_mem3 = measure_train_test(
    cnn3, x_train_cnn, y_train_cat, x_test_cnn, y_test_cat, epochs=3,␣
 ↪batch_size=128, validation_split=0.1
)
```

```
print(f'CNN 3 Test accuracy: {score3[1]:.4f}')
print(f'CNN 3 Train time: {train_time3:.2f}s, Train peak mem: {train_mem3:.
  ↪2f}MB')
print(f'CNN 3 Test time: {test_time3:.2f}s, Test peak mem: {test_mem3:.2f}MB')
```

```
Epoch 1/3
422/422             19s 41ms/step -
accuracy: 0.8852 - loss: 0.3983 - val_accuracy: 0.9853 - val_loss: 0.0557
Epoch 2/3
422/422             17s 41ms/step -
accuracy: 0.9852 - loss: 0.0490 - val_accuracy: 0.9875 - val_loss: 0.0472
Epoch 3/3
422/422             18s 42ms/step -
accuracy: 0.9910 - loss: 0.0290 - val_accuracy: 0.9903 - val_loss: 0.0410
CNN 3 Test accuracy: 0.9877
CNN 3 Train time: 53.90s, Train peak mem: 3.03MB
CNN 3 Test time: 1.57s, Test peak mem: 0.09MB
```

[14]:
```
import pandas as pd

# Collect architecture and performance data
models_info = [
    {
        'Model': 'CNN 1',
        'Test Accuracy': score1[1],
        'Parameters': cnn1.count_params(),
        'Layers': len(cnn1.layers),
        'Train Time (s)': train_time1,
        'Train Peak Mem (MB)': train_mem1,
        'Test Time (s)': test_time1,
        'Test Peak Mem (MB)': test_mem1
    },
    {
        'Model': 'CNN 2',
        'Test Accuracy': score2[1],
        'Parameters': cnn2.count_params(),
        'Layers': len(cnn2.layers),
        'Train Time (s)': train_time2,
        'Train Peak Mem (MB)': train_mem2,
        'Test Time (s)': test_time2,
        'Test Peak Mem (MB)': test_mem2
    },
    {
        'Model': 'CNN 3',
        'Test Accuracy': score3[1],
        'Parameters': cnn3.count_params(),
        'Layers': len(cnn3.layers),
```

```
        'Train Time (s)': train_time3,
        'Train Peak Mem (MB)': train_mem3,
        'Test Time (s)': test_time3,
        'Test Peak Mem (MB)': test_mem3
    }
]

df_models = pd.DataFrame(models_info)
print(df_models)
```

```
   Model  Test Accuracy  Parameters  Layers  Train Time (s)  \
0  CNN 1         0.9828      347146       5       19.076913
1  CNN 2         0.9823     1386506       7       44.853460
2  CNN 3         0.9877     1192042       6       53.901463

   Train Peak Mem (MB)  Test Time (s)  Test Peak Mem (MB)
0             2.445415       0.764529            0.089445
1             2.754482       1.134619            0.084620
2             3.026632       1.569531            0.090665
```